

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «Информатика и системы управления»

КАФЕДРА ИУ7 «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЁТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Тема курсовой работы»

Студент ИУ7-51Б _____ И.И. Иванов
(Группа) (Подпись, дата) (И.О. Фамилия)

Руководитель курсовой работы _____ П.П. Петров
(Подпись, дата) (И.О. Фамилия)

Москва, 2026

РЕФЕРАТ

Расчётно-пояснительная записка 25 с., 4 рис., 5 табл., 10 источн., 4 прил.

УМНОЖЕНИЕ МАТРИЦ, АЛГОРИТМ ВИНОГРАДА, ТРУДОЁМКОСТЬ АЛГОРИТМА, МОДЕЛЬ ВЫЧИСЛЕНИЙ, ЗАМЕР ВРЕМЕНИ, C++

Объектом исследования являются алгоритмы умножения матриц: классический алгоритм и алгоритм Винограда в простой и оптимизированной реализациях.

Цель работы — сравнить эти алгоритмы по трудоёмкости и по времени выполнения на матрицах разного размера.

В аналитической части рассмотрены постановка задачи и идеи алгоритмов. В конструкторской части приведены требования к программе, схемы алгоритмов и оценки их трудоёмкости в принятой модели вычислений. В технологической части описаны средства реализации, исходный код и функциональные тесты. В исследовательской части приведены результаты замеров времени и их сравнение с теоретическими оценками.

Оптимизированный алгоритм Винограда на квадратных матрицах размером 500 оказался быстрее классического алгоритма примерно на 28 %, что согласуется с оценкой трудоёмкости. Результаты работы применимы при выборе алгоритма умножения матриц в задачах, где матрицы помещаются в оперативную память.

СОДЕРЖАНИЕ

РЕФЕРАТ **2**

ВВЕДЕНИЕ **5**

1 Аналитическая часть **6**

1.1 Постановка задачи 6

1.2 Классический алгоритм 6

1.3 Алгоритм Винограда 6

1.3.1 Предварительная обработка 7

1.3.2 Вычисление элементов результата 7

1.3.3 Оптимизации алгоритма 8

1.4 Алгоритм Штрассена 8

2 Конструкторская часть **10**

2.1 Требования к программе 10

2.2 Модель вычислений 10

2.3 Схемы алгоритмов 10

2.4 Оценка трудоёмкости 12

3 Технологическая часть **14**

3.1 Средства реализации 14

3.2 Реализация алгоритмов 14

3.3 Функциональное тестирование 15

4 Исследовательская часть **18**

4.1 Технические характеристики 18

4.2 Время выполнения 18

ЗАКЛЮЧЕНИЕ **21**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **22**

ПРИЛОЖЕНИЕ А **23**

ПРИЛОЖЕНИЕ Б **24**

Б.1 Параметры замеров 24

Б.2 Схема программы замеров 24

ВВЕДЕНИЕ

Умножение матриц — одна из базовых операций линейной алгебры [1]. К нему сводятся решение систем линейных уравнений, преобразования координат в компьютерной графике, обработка изображений и вычисления в нейронных сетях. Поэтому время умножения матриц заметно влияет на производительность многих программ.

Классический алгоритм вычисляет каждый элемент результата как скалярное произведение строки первой матрицы на столбец второй и требует порядка n^3 умножений для квадратных матриц размером n [2]. Существуют алгоритмы, которые уменьшают число умножений за счёт дополнительных сложений и предварительной обработки данных. Один из них — алгоритм Винограда [3], асимптотически не отличающийся от классического, но выполняющий примерно вдвое меньше умножений.

На практике выигрыш от такой замены зависит от соотношения стоимостей операций на конкретном процессоре, от организации доступа к памяти и от того, насколько удачно реализация избавлена от лишних вычислений. Поэтому теоретическую оценку полезно дополнить замерами времени.

Целью работы является сравнение классического алгоритма умножения матриц и алгоритма Винограда по трудоёмкости и по времени выполнения.

Для достижения поставленной цели необходимо решить следующие задачи:

1) описать классический алгоритм умножения матриц и алгоритм Винограда;

2) разработать схемы алгоритмов и оценить их трудоёмкость в выбранной модели вычислений;

3) реализовать алгоритмы и проверить их на функциональных тестах;

4) замерить время выполнения реализаций на матрицах разного размера;

5) сравнить результаты замеров с теоретическими оценками и сделать выводы о применимости алгоритмов.

1 Аналитическая часть

1.1 Постановка задачи

Пусть даны матрица A размером $m \times n$ и матрица B размером $n \times q$. Их произведением называется матрица C размером $m \times q$, элементы которой вычисляются по формуле (1.1) [1, с. 23]:

$$c_{ij} = \sum_{k=1}^{n} a_{ik} \cdot b_{kj}, \quad i = 1, ..., m, \quad j = 1, ..., q, \tag{1.1}$$

где a_{ik} — элемент матрицы A в строке i и столбце k, b_{kj} — элемент матрицы B в строке k и столбце j. Произведение определено только тогда, когда число столбцов первой матрицы равно числу строк второй.

Умножение матриц некоммутативно, то есть в общем случае $AB \neq BA$. Поэтому алгоритм должен сохранять порядок сомножителей, а программа — проверять согласованность размеров до начала вычислений.

1.2 Классический алгоритм

Классический алгоритм вычисляет элементы матрицы C непосредственно по формуле (1.1) тремя вложенными циклами [2]. Внешние циклы перебирают строки матрицы A и столбцы матрицы B, а внутренний цикл накапливает скалярное произведение строки на столбец.

Для вычисления одного элемента требуется n умножений и n сложений, а всего алгоритм выполняет mnq умножений. Для квадратных матриц размером n это составляет n^3 умножений, и асимптотическая сложность алгоритма равна $O(n^3)$.

1.3 Алгоритм Винограда

Алгоритм Винограда основан на наблюдении, что скалярное произведение двух векторов можно выразить через попарные суммы их элементов [3]. Часть слагаемых при этом зависит только от строки первой матрицы или только от столбца второй, и её можно вычислить заранее один раз [4].

Каждая строка матрицы A участвует в вычислении q элементов результата, а каждый столбец матрицы B — в вычислении m элементов. Поэтому величины, зависящие только от строки или только от столбца, выгодно вычислить один раз и использовать многократно. На этом и строится экономия умножений: вычисление разбивается на предварительную обработку и основной этап, описанные в следующих пунктах.

1.3.1 Предварительная обработка

Пусть $d = \lfloor n/2 \rfloor$. Для каждой строки i матрицы A и каждого столбца j матрицы B заранее вычисляются величины, приведённые в формулах (1.2):

$$r_i = \sum_{k=1}^{d} a_{i,2k-1} \cdot a_{i,2k}, \quad s_j = \sum_{k=1}^{d} b_{2k-1,j} \cdot b_{2k,j}, \tag{1.2}$$

где r_i — множитель строки i, s_j — множитель столбца j. На их вычисление требуется $d(m+q)$ умножений, что для больших матриц мало по сравнению с основной частью алгоритма.

Множители хранятся в двух вспомогательных массивах длиной m и q. Дополнительная память, таким образом, линейна по размерам матриц и не зависит от их внутренней размерности n. Если n нечётно, последний столбец матрицы A и последняя строка матрицы B в вычислении множителей не участвуют и обрабатываются отдельно на основном этапе.

1.3.2 Вычисление элементов результата

Элемент матрицы C вычисляется по формуле (1.3):

$$c_{ij} = \sum_{k=1}^{d} \left(a_{i,2k-1} + b_{2k,j}\right) \cdot \left(a_{i,2k} + b_{2k-1,j}\right) - r_i - s_j. \tag{1.3}$$

Раскрытие скобок даёт четыре слагаемых. Два из них образуют искомое скалярное произведение, а два других совпадают с r_i и s_j и вычитаются. Если

n нечётно, к каждому элементу дополнительно прибавляется произведение $a_{in} \cdot b_{nj}$, которое не вошло ни в одну пару.

В итоге на один элемент приходится d умножений вместо n, то есть вдвое меньше, а число сложений возрастает. Выигрыш во времени получается тогда, когда умножение выполняется заметно дольше сложения.

1.3.3 Оптимизации алгоритма

Простая реализация алгоритма Винограда выполняет лишние вычисления, которые можно убрать без изменения результата:

— накапливать сумму во временной переменной, а не в элементе матрицы, что сокращает число обращений к памяти;

— заменить вычисление индексов $2k-1$ и $2k$ шагом цикла, равным двум;

— вычитать множители строки и столбца один раз до цикла по k, а не после него;

— обрабатывать нечётную размерность отдельным циклом, который выполняется только при необходимости, а именно:

а) проверять чётность n один раз до основных циклов;

б) прибавлять последнее произведение в том же проходе по элементам результата.

1.4 Алгоритм Штрассена

Алгоритм Штрассена делит каждую матрицу на четыре блока и вычисляет произведение через семь умножений блоков вместо восьми [5]. Рекурсивное применение этого приёма даёт сложность $O\big(n^{\log_2 7}\big) \approx O\big(n^{2.81}\big)$.

На практике алгоритм выигрывает только на больших матрицах, потому что требует дополнительной памяти под промежуточные блоки и большого числа сложений [6]. В работе он не реализуется и упомянут для сравнения.

Вывод

Классический алгоритм вычисляет произведение по определению и выполняет mnq умножений. Алгоритм Винограда за счёт предварительной обработки выполняет вдвое меньше умножений ценой дополнительных сложений, а его

простая реализация допускает ряд оптимизаций. Для сравнения выбраны классический алгоритм и две реализации алгоритма Винограда.

2 Конструкторская часть

2.1 Требования к программе

Программа должна поддерживать два режима работы:

— функциональный режим, в котором:

а) пользователь вводит размеры и элементы двух матриц;

б) программа проверяет согласованность размеров;

в) программа выводит результат умножения каждым из трёх алгоритмов;

— режим замеров, в котором программа умножает случайные квадратные матрицы заданных размеров и выводит среднее время работы каждого алгоритма.

При несогласованных размерах матриц программа должна выводить сообщение об ошибке и не выполнять умножение.

2.2 Модель вычислений

Для оценки трудоёмкости используется следующая модель вычислений:

1) операции `+`, `-`, `=`, `+=`, `-=`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `[]`, `++` и `--` имеют стоимость 1;

2) операции `*`, `/` и `%` имеют стоимость 2;

3) стоимость условного оператора равна стоимости вычисления условия плюс стоимость выполняемой ветви;

4) стоимость цикла `for` вычисляется по формуле (2.1).

$$f_{\text{for}} = f_{\text{init}} + f_{\text{cmp}} + N\left(f_{\text{body}} + f_{\text{inc}} + f_{\text{cmp}}\right), \tag{2.1}$$

где f_{init} — стоимость инициализации счётчика, f_{cmp} — стоимость сравнения, f_{inc} — стоимость приращения счётчика, f_{body} — стоимость тела цикла, N — число итераций.

2.3 Схемы алгоритмов

Схемы алгоритмов выполнены по ГОСТ 19.701-90 [7]. Укрупнённые схемы классического алгоритма и алгоритма Винограда приведены на рисунках 2.1–2.2.

Обе схемы начинаются с ввода матриц и заканчиваются выводом результата, а различаются тем, что происходит между ними. В классическом алгоритме каждый

элемент результата вычисляется независимо от остальных в одном проходе по строкам и столбцам. В алгоритме Винограда перед основным циклом выполняется предварительная обработка, а внутри цикла к сумме попарных произведений добавляется учёт нечётной размерности. Проверка согласованности размеров матриц на схемах не показана, она выполняется до вызова алгоритмов.

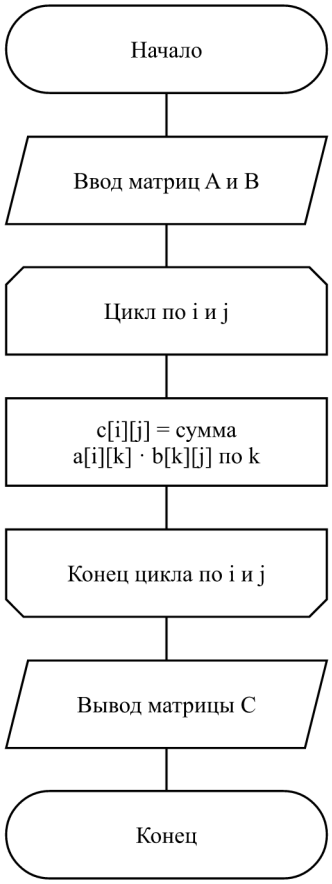


Рисунок 2.1 — Укрупнённая схема классического алгоритма умножения матриц

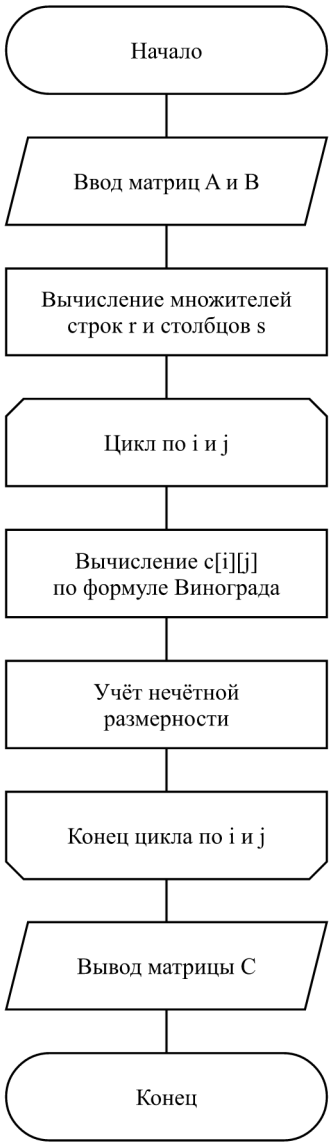


Рисунок 2.2 — Укрупнённая схема алгоритма Винограда

2.4 Оценка трудоёмкости

Пусть матрица A имеет размер $M \times N$, а матрица B — размер $N \times Q$. Тело внутреннего цикла классического алгоритма содержит четыре индексации по два обращения, умножение и сложение с присваиванием, и его стоимость равна 9. Применение формулы (2.1) к трём вложенным циклам даёт трудоёмкость классического алгоритма, приведённую в формуле (2.2):

$$f_{\text{classic}} = 11MNQ + 4MQ + 4M + 2. \tag{2.2}$$

В простой реализации алгоритма Винограда тело внутреннего цикла содержит вычисление индексов $2k$ и $2k + 1$ и стоит 25, а цикл выполняется $N/2$ раз.

В оптимизированной реализации индексы заменены шагом цикла, сумма накапливается во временной переменной, и стоимость тела снижается до 15. Старшие члены трудоёмкости алгоритмов сведены в таблицу 2.1.

Таблица 2.1 — Старшие члены трудоёмкости алгоритмов

Алгоритм	Старший член	Отношение к классическому
Классический	$11MNQ$	1,00
Виноград, простая реализация	$13{,}5\ MNQ$	1,23
Виноград, оптимизированная реализация	$8{,}5\ MNQ$	0,77

Предварительная обработка в алгоритме Винограда имеет трудоёмкость порядка $N(M+Q)$ и при больших размерах матриц пренебрежимо мала.

Вывод

Сформулированы требования к программе, приведены схемы алгоритмов и получены оценки трудоёмкости. В принятой модели вычислений простая реализация алгоритма Винограда уступает классическому алгоритму из-за дорогого вычисления индексов, а оптимизированная реализация выполняет примерно на 23 % меньше операций.

3 Технологическая часть

3.1 Средства реализации

Для реализации выбран язык C++ стандарта C++20 [8]. Он компилируется в машинный код без промежуточной виртуальной машины, поэтому замеры отражают стоимость самих алгоритмов, а не накладные расходы среды выполнения. Матрицы хранятся как векторы строк из стандартной библиотеки.

Время измеряется монотонными часами `std::chrono::steady_clock` из стандартной библиотеки, показания которых не зависят от перевода системного времени. Программа собирается компилятором GCC с уровнем оптимизации `-O2`, при котором компилятор выполняет большинство оптимизаций, не увеличивающих размер кода [9].

3.2 Реализация алгоритмов

Реализация классического алгоритма приведена на листинге 3.1, а оптимизированная реализация алгоритма Винограда — на листинге 3.2. Определения типов `Matrix` и `MatrixRow` вынесены в приложение А и приведены на листинге А.1.

```
 1 Matrix multiplyClassic(const Matrix &matrixA, const Matrix &matrixB) {
 2   size_t nRowsA = matrixA.size();
 3   size_t nColsA = matrixA[0].size();
 4   size_t nColsB = matrixB[0].size();
 5   Matrix result(nRowsA, MatrixRow(nColsB, 0));
 6
 7   for (size_t i = 0; i < nRowsA; ++i) {
 8     for (size_t j = 0; j < nColsB; ++j) {
 9       for (size_t k = 0; k < nColsA; ++k) {
10         result[i][j] += matrixA[i][k] * matrixB[k][j];
11       }
12     }
13   }
14
15   return result;
16 }
```

Листинг 3.1 — Классический алгоритм умножения матриц

В оптимизированной реализации множители строк и столбцов вычисляются до основного цикла, сумма накапливается в локальной переменной `sum`, а произ-

ведение для нечётной размерности прибавляется в том же проходе по элементам результата.

```
Matrix multiplyWinograd(const Matrix &matrixA, const Matrix &matrixB) {
  size_t nRows = matrixA.size();
  size_t nInner = matrixB.size();
  size_t nCols = matrixB[0].size();
  size_t half = nInner / 2;

  std::vector<int> rowFactor(nRows, 0);
  for (size_t i = 0; i < nRows; ++i) {
    for (size_t k = 0; k < half * 2; k += 2) {
      rowFactor[i] += matrixA[i][k] * matrixA[i][k + 1];
    }
  }

  std::vector<int> colFactor(nCols, 0);
  for (size_t j = 0; j < nCols; ++j) {
    for (size_t k = 0; k < half * 2; k += 2) {
      colFactor[j] += matrixB[k][j] * matrixB[k + 1][j];
    }
  }

  Matrix result(nRows, MatrixRow(nCols, 0));
  for (size_t i = 0; i < nRows; ++i) {
    for (size_t j = 0; j < nCols; ++j) {
      int sum = -rowFactor[i] - colFactor[j];
      for (size_t k = 0; k < half * 2; k += 2) {
        sum += (matrixA[i][k] + matrixB[k + 1][j]) *
               (matrixA[i][k + 1] + matrixB[k][j]);
      }
      if (nInner % 2 != 0) {
        sum += matrixA[i][nInner - 1] * matrixB[nInner - 1][j];
      }
      result[i][j] = sum;
    }
  }

  return result;
}
```

Листинг 3.2 — Оптимизированный алгоритм Винограда

3.3 Функциональное тестирование

Тесты составлены по методу чёрного ящика и охватывают квадратные и прямоугольные матрицы, нечётную внутреннюю размерность, вырожденные случаи и несогласованные размеры. Тесты приведены в таблице 3.1. Все три реализации прошли все тесты.

Таблица 3.1 — Функциональные тесты

№ теста	Описание	Входные данные	Ожидаемый результат
1	Квадратные матрицы размером 3×3	$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$ $B = \begin{pmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{pmatrix}$	$C = \begin{pmatrix} 30 & 24 & 18 \\ 84 & 69 & 54 \\ 138 & 114 & 90 \end{pmatrix}$
2	Прямоугольные матрицы размером 2×3 и 3×2	$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$ $B = \begin{pmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{pmatrix}$	$C = \begin{pmatrix} 58 & 64 \\ 139 & 154 \end{pmatrix}$
3	Строка на столбец, нечётная внутренняя размерность	$A = (2 \ \ 3 \ \ 4)$ $B = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$	$C = (20)$
4	Матрицы размером 1×1 с отрицательным элементом	$A = (5)$ $B = (-3)$	$C = (-15)$
5	Единичная матрица слева	$A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ $B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$	$C = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$
6	Нулевая матрица слева	$A = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$ $B = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$	$C = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$
7	Несогласованные размеры 2×3 и 2×2	$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$ $B = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$	Сообщение об ошибке
8	Матрица-столбец на матрицу-строку	$A = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$ $B = (3 \ \ 4)$	$C = \begin{pmatrix} 3 & 4 \\ 6 & 8 \end{pmatrix}$

Продолжение таблицы 3.1

№ теста	Описание	Входные данные	Ожидаемый результат
9	Матрицы с отрицательными элементами	$A = \begin{pmatrix} -1 & 2 \\ 3 & -4 \end{pmatrix}$ $B = \begin{pmatrix} 2 & 0 \\ 1 & -1 \end{pmatrix}$	$C = \begin{pmatrix} 0 & -2 \\ 2 & 4 \end{pmatrix}$

Вывод

Алгоритмы реализованы на языке C++ и проверены на функциональных тестах, включающих нечётную размерность и несогласованные размеры матриц. Все реализации дают одинаковые результаты, совпадающие с ожидаемыми.

4 Исследовательская часть

4.1 Технические характеристики

Замеры выполнялись на ноутбуке со следующими характеристиками:

— операционная система Ubuntu 24.04;

— процессор AMD Ryzen 5 5600H с частотой 3,3 ГГц;

— оперативная память объёмом 16 ГБ.

Во время замеров ноутбук был подключён к сети питания, а из пользовательских программ работала только программа замеров.

Частота процессора в таком режиме не снижается ради экономии энергии, а посторонние программы не отнимают у программы замеров процессорное время. Это уменьшает разброс измерений между повторами. Остальные параметры замеров, в том числе версия компилятора и уровень оптимизации, приведены в приложении Б.

4.2 Время выполнения

Замеры проводились на случайных квадратных матрицах размером от 100 до 500 с шагом 100. Для каждого размера умножение повторялось 10 раз, и в таблицу заносилось среднее время. Параметры замеров перечислены в таблице Б.1 приложения Б. Результаты приведены в таблице 4.1 и на рисунке 4.1, графики построены библиотекой Lilaq [10].

Таблица 4.1 — Время умножения квадратных матриц

Размер	**Классический, мс**	**Виноград, мс**	**Виноград оптимизир., мс**
100	14	2	5
200	19	42	17
300	144	90	59
400	238	241	220
500	545	502	390

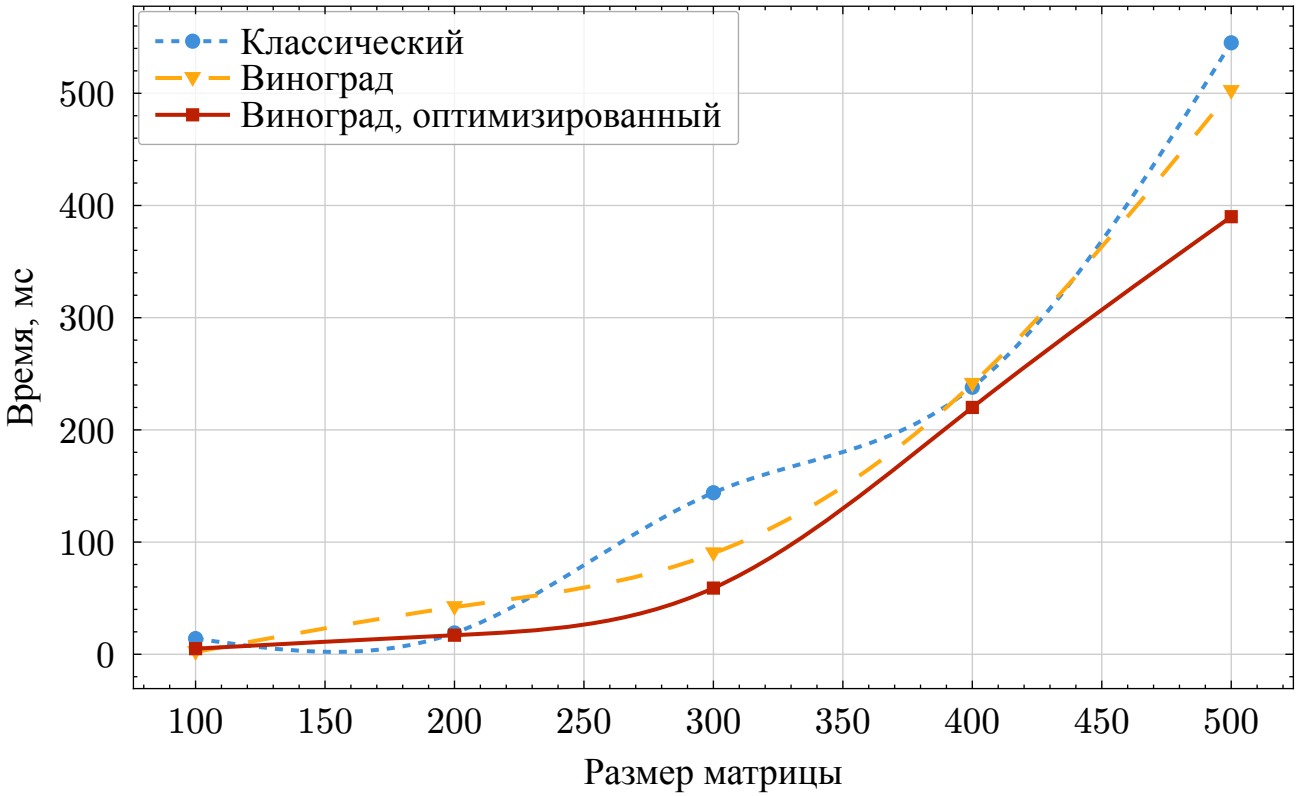


Рисунок 4.1 — Зависимость времени умножения от размера квадратных матриц

На матрицах размером 500 оптимизированный алгоритм Винограда работает 390 мс против 545 мс у классического алгоритма, то есть быстрее примерно на 28 %. Это близко к оценке из таблицы 2.1, по которой оптимизированная реализация выполняет на 23 % меньше операций.

Простая реализация алгоритма Винограда, вопреки оценке, оказалась немного быстрее классического алгоритма. Модель вычислений считает вычисление индексов $2k$ и $2k + 1$ отдельными операциями, тогда как компилятор при оптимизации заменяет его приращением указателя. Поэтому модель переоценивает стоимость простой реализации.

На матрицах размером 100 и 200 время работы составляет единицы и десятки миллисекунд и заметно зависит от случайных задержек, поэтому различия между алгоритмами на таких размерах нельзя считать значимыми.

Вывод

Оптимизированный алгоритм Винограда оказался самым быстрым из рассмотренных и на матрицах размером 500 опередил классический алгоритм примерно на 28 %. Отношение времён согласуется с оценкой трудоёмкости для

оптимизированной реализации, а для простой реализации модель вычислений даёт завышенную оценку.

ЗАКЛЮЧЕНИЕ

Цель работы, состоявшая в сравнении классического алгоритма умножения матриц и алгоритма Винограда по трудоёмкости и по времени выполнения, достигнута.

В ходе работы решены все поставленные задачи. Описаны классический алгоритм и алгоритм Винограда, а также рассмотрены возможные оптимизации последнего. Разработаны схемы алгоритмов и получены оценки их трудоёмкости в модели вычислений, учитывающей разную стоимость арифметических операций. Алгоритмы реализованы на языке C++ и проверены на функциональных тестах.

Замеры времени показали, что оптимизированный алгоритм Винограда на матрицах размером 500 работает примерно на 28 % быстрее классического алгоритма, что согласуется с теоретической оценкой. Простая реализация алгоритма Винограда даёт выигрыш лишь около 8 %, поэтому без оптимизаций применять её нецелесообразно.

Для матриц размером порядка сотни и меньше выбор алгоритма практически не влияет на время работы, и предпочтительнее более простой классический алгоритм. Для больших матриц стоит использовать оптимизированный алгоритм Винограда, а при дальнейшем росте размеров — рассмотреть блочные алгоритмы, лучше использующие кеш процессора.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Веретенников В. Н. Практикум по линейной алгебре: учеб. пособие. — Санкт-Петербург : РГГМУ, 2015. — 140 с.

2. Кормен Т. и др. Алгоритмы: построение и анализ. 3-е изд. — Москва : Вильямс, 2013. — 1328 с.

3. Winograd S. A new algorithm for inner product // IEEE Transactions on Computers. — 1968. — Т. C-17, № 7. — С. 693–694.

4. Кнут Д. Э. Искусство программирования. Т. 2. Получисленные алгоритмы. 3-е изд. — Москва : Вильямс, 2007. — 832 с.

5. Strassen V. Gaussian elimination is not optimal // Numerische Mathematik. — 1969. — Т. 13, № 4. — С. 354–356.

6. Golub G. H., Van Loan C. F. Matrix Computations. 4th ed. — Baltimore : Johns Hopkins University Press, 2013. — 756 с.

7. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения. — Москва : Стандартинформ, 2010. — 23 с.

8. ISO/IEC 14882:2020. Programming languages — C++ [Электронный ресурс]. — Режим доступа: https://www.iso.org/standard/79358.html (дата обращения: 20.09.2026).

9. Using the GNU Compiler Collection (GCC). Options That Control Optimization [Электронный ресурс]. — Режим доступа: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html (дата обращения: 20.09.2026).

10. Lilaq. Advanced data visualization in Typst [Электронный ресурс]. — Режим доступа: https://lilaq.org/ (дата обращения: 20.09.2026).

ПРИЛОЖЕНИЕ А

В приложении приведены определения типов, которыми программа представляет матрицы. Матрица хранится как вектор строк, а каждая строка — как вектор целых чисел. Такое представление позволяет обращаться к элементу через двойную индексацию и не требует ручного управления памятью.

Определения типов приведены на листинге А.1. Оба алгоритма принимают матрицы по константной ссылке, чтобы не копировать их при вызове, и возвращают результат по значению.

```
1 #include <vector>
2
3 using MatrixRow = std::vector<int>;
4 using Matrix = std::vector<MatrixRow>;
```

Листинг А.1 — Определения типов матрицы

ПРИЛОЖЕНИЕ Б

Б.1 Параметры замеров

Параметры, при которых получены результаты исследовательской части, перечислены в таблице Б.1. Они нужны для того, чтобы замеры можно было повторить на другом оборудовании и сравнить результаты.

Параметр	**Значение**
Размеры матриц	от 100 до 500 с шагом 100
Число повторов	10
Элементы матриц	случайные целые числа от −100 до 100
Часы	`std::chrono::steady_clock`
Компилятор и оптимизация	GCC, `-O2`

Таблица Б.1 — Параметры замеров времени

Перед каждым замером программа заново заполняет матрицы случайными числами, чтобы результат не зависел от содержимого кеша, оставшегося от предыдущего повтора. Время заполнения в замер не входит, поскольку часы запускаются непосредственно перед вызовом алгоритма и останавливаются сразу после него.

Б.2 Схема программы замеров

Укрупнённая схема программы замеров приведена на рисунке Б.1. Внешний цикл перебирает размеры матриц, а внутренний повторяет умножение заданное число раз и накапливает суммарное время.



Рисунок Б.1 — Укрупнённая схема программы замеров

После завершения внутреннего цикла суммарное время делится на число повторов, и среднее значение выводится вместе с размером матриц. Такой вывод программы без изменений переносится в CSV-файлы, по которым строятся таблица и график исследовательской части.